

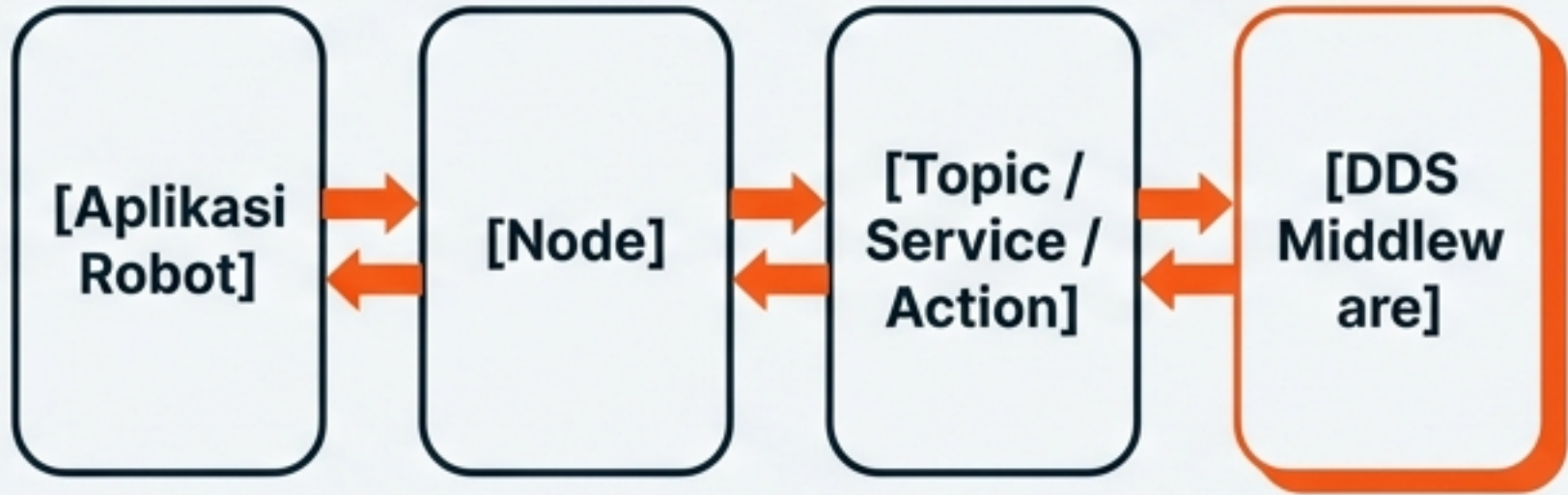
Modul 04: ROS2 Dasar dan Simulasi Gazebo

Membangun Fondasi Perangkat Lunak Robotika

ROS1 vs ROS2

Middleware	Custom (TCPROS)	DDS (Data Distribution Service)
Arsitektur	Terpusat (ROS Master)	Terdistribusi (Tanpa Master)
Kinerja	Standar	Mendukung Real-Time & Multi-robot
Keamanan	Tidak Ada	Terintegrasi (DDS Security)

Arsitektur Ekosistem ROS2



5 Pilar Konsep Dasar Komunikasi ROS2



Node: Unit komputasi tunggal (satu proses).



Topic: Komunikasi async publisher-subscriber.



Service: Komunikasi sync request-response.



Action: Goal-feedback-result untuk tugas panjang.



Parameter: Konfigurasi variabel saat runtime.



```
ros2 node list
ros2 topic list
ros2 topic echo /nama_topik
ros2 service list
ros2 param list
```


Struktur Package ROS2 & Python Launch File

ament_cmake_package

CMakeLists.txt

package.xml

launch

urdf

worlds

config

scripts

```
def generate_launch_description():  
    return LaunchDescription(...)
```

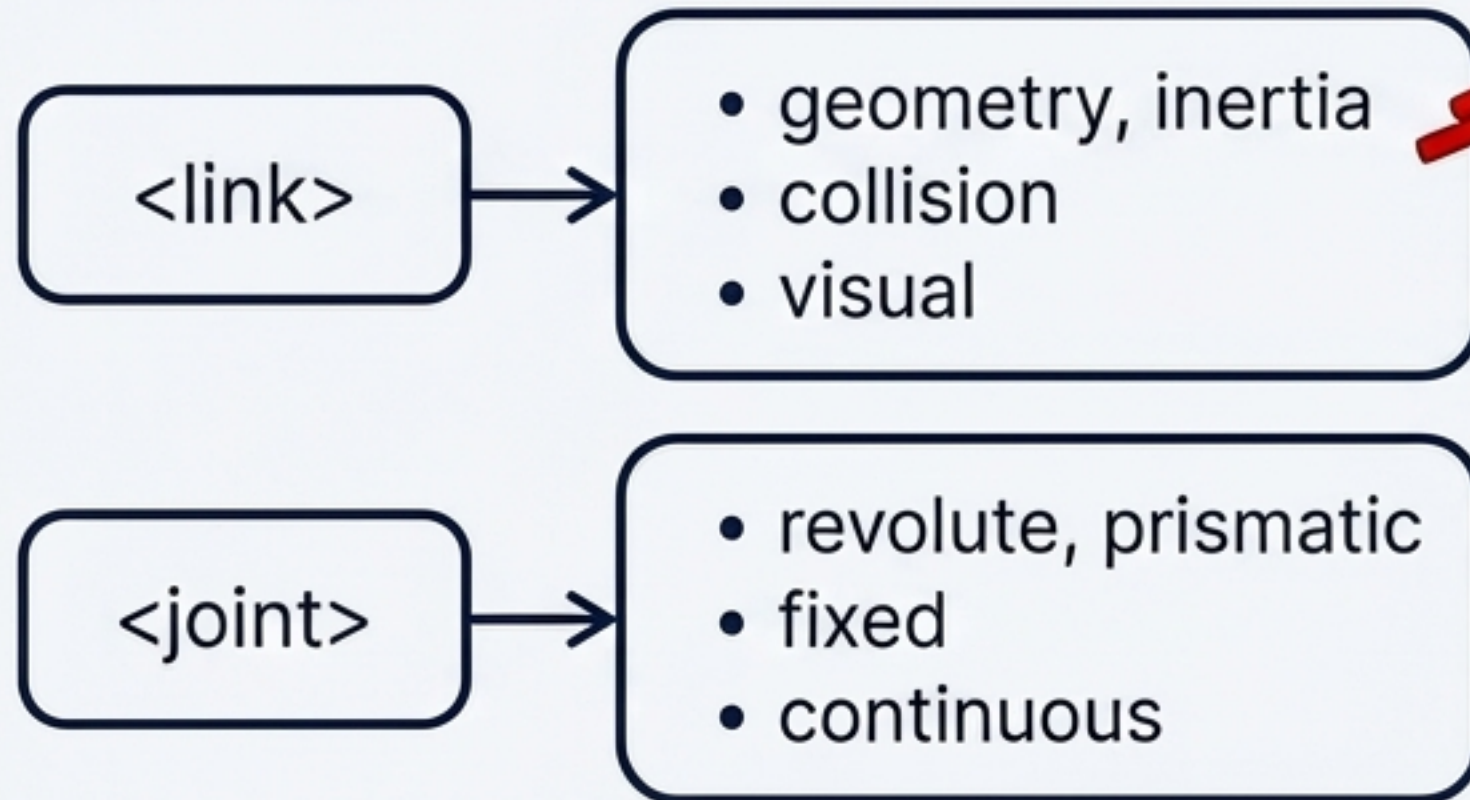
Highlight: Penggunaan TimerAction sangat krusial untuk menambahkan delay bertingkat saat memuat node.

Proses Build Workspace

```
colcon build --packages-select gazebo_praktikum  
source install/setup.bash
```


Membangun Anatomi Robot: URDF dan XACRO

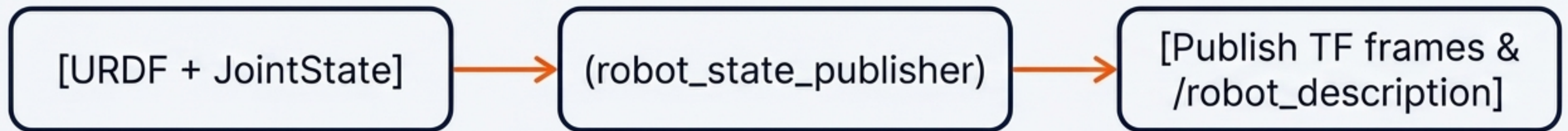
URDF (Unified Robot Description Format)



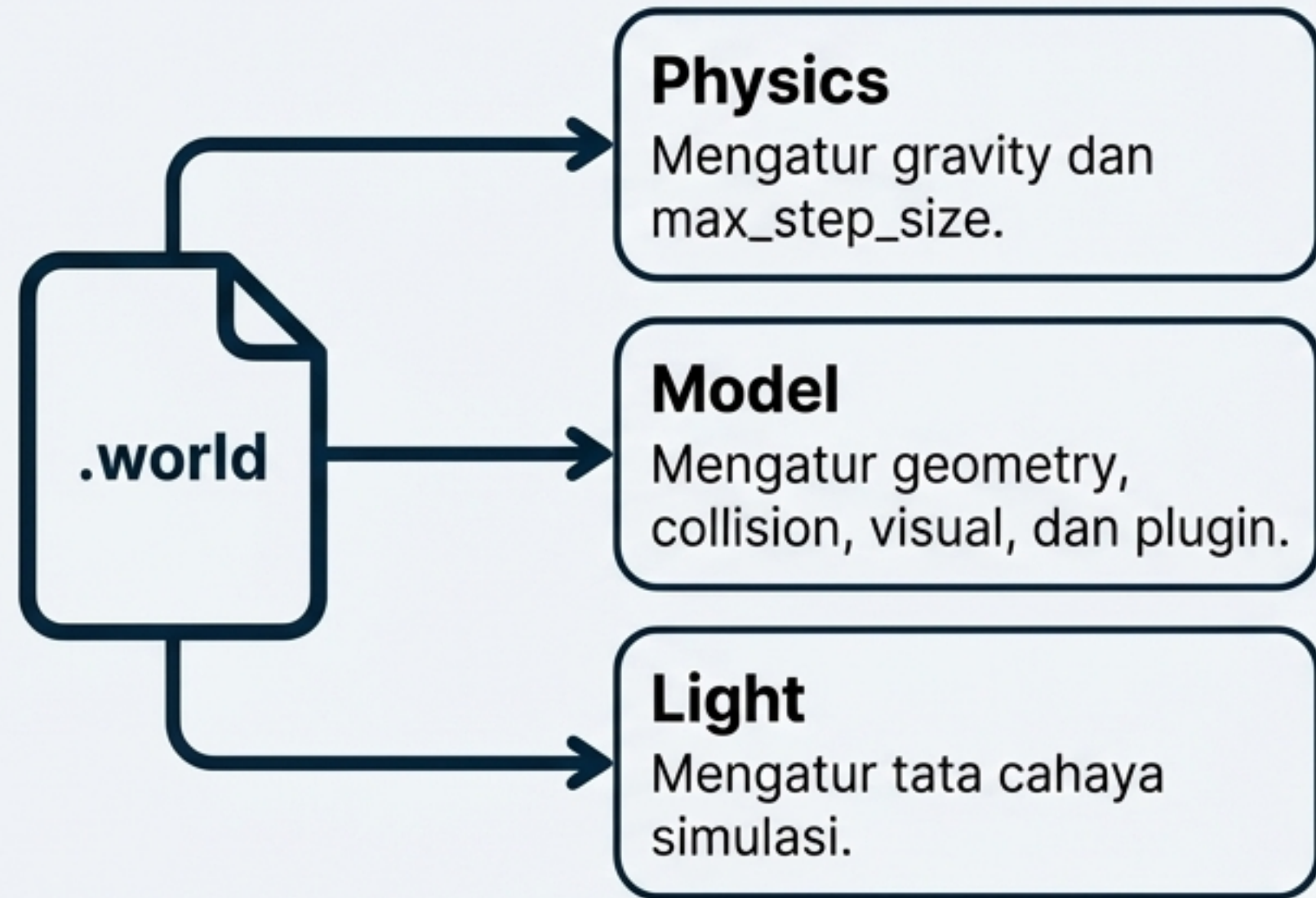
XACRO (XML Macros)

Ekstensi URDF menggunakan **macro** dan **property** agar kode dapat didaur ulang.

```
xacro robot.xacro > robot.urdf
```



Lingkungan Simulasi: Format SDF dan Gazebo Classic



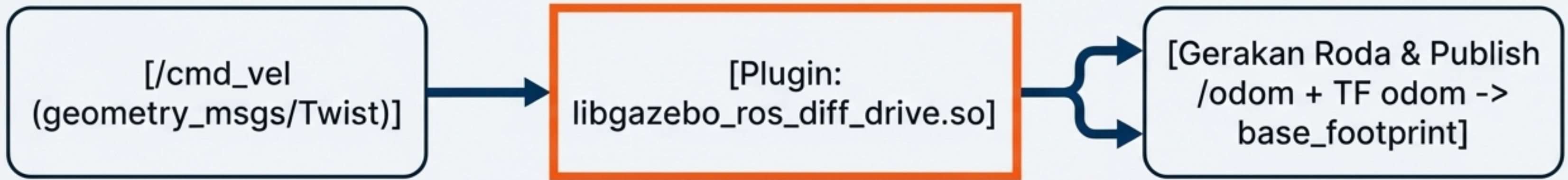
SDF vs URDF

SDF	URDF
Kompleks, khusus untuk simulasi fisika lengkap, mengatur lingkungan & objek bebas.	Standar ekosistem ROS, khusus mendeskripsikan struktur hierarkis internal robot.



Catatan Kestabilan: Praktikum ini menggunakan Gazebo Classic 11 yang sangat stabil berpasangan dengan ROS2 Humble.

Integrasi Pergerakan: Gazebo Plugin Roda Diferensial



XACRO Parameter

```
<gazebo>
  <plugin name='diff_drive' filename='libgazebo_ros_diff_drive.so'>
    <wheel_separation>0.2</wheel_separation>
    <wheel_radius>0.05</wheel_radius>
    <publish_odom_tf>true</publish_odom_tf>
  </plugin>
</gazebo>
```


Sistem Persepsi: Plugin Sensor Kamera dan LIDAR



Sensor Visual (Kamera)

Plugin:

```
libgazebo_ros_camera.so
```

Topik Output:

/camera/image_raw
(sensor_msgs/Image) dan
/camera/camera_info.



Sensor Spasial (LIDAR)

Plugin:

```
libgazebo_ros_ray_sensor.so
```

Topik Output:

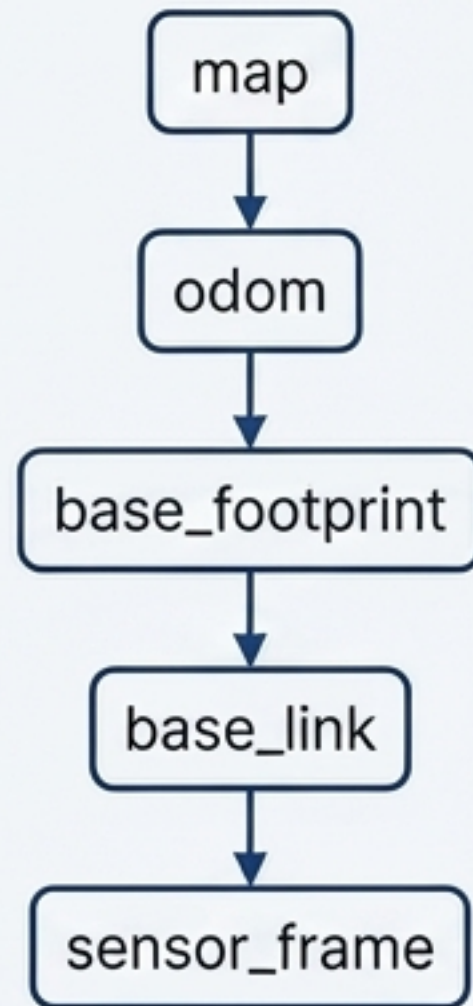
/scan (sensor_msgs/LaserScan).

Parameter Kritis:

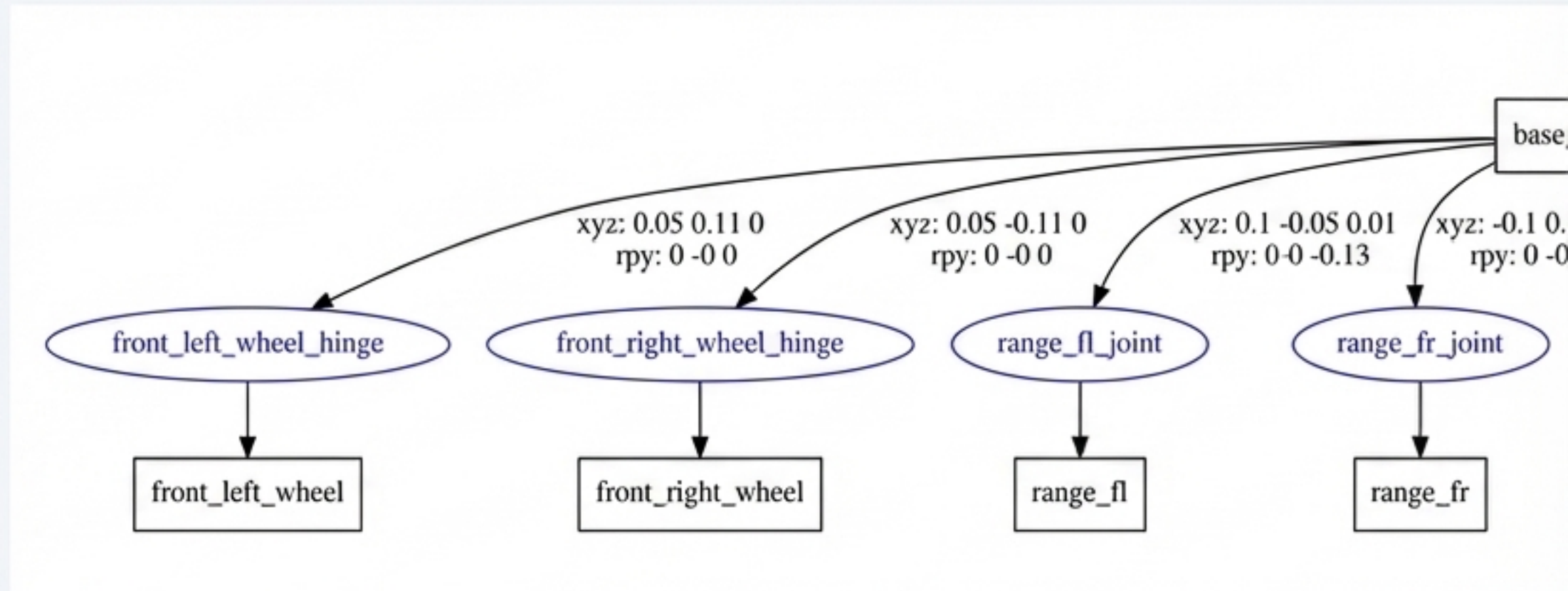
min_range, **mix_range**, **max_range**,
samples (360 nilai untuk coverage 360 derajat), dan **update_rate** (Hz).

Catatan Tambahan: Konfigurasi dibungkus dalam tag `<sensor>` pada file XACRO. Kedua output sensor ini dapat dimonitor secara visual melalui aplikasi RViz2.

Melacak Koordinat: Sistem TF2 Frame Tree



Dikelola waktu-nyata (real-time) oleh robot_state_publisher yang membaca struktur dari URDF.



Perintah Debugging

```
ros2 run tf2_tools view_frames
```

```
ros2 run tf2_ros tf2_echo odom base_link
```

Callout Troubleshooting: Masalah TF yang paling sering dijumpai adalah error 'Frame not found', biasanya karena URDF terputus atau node belum tereksekusi penuh.

Percobaan 1 – Pengenalan Antarmuka Empty World



Kebutuhan Sistem:
1 Terminal

```
ros2 launch gazebo_praktikum percobaan1_empty_world.launch.py
```

(Argumen default: gui:=true, verbose:=false, paused:=false)

Checklist Observasi (Tujuan Praktikum)

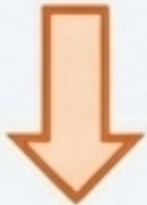
- ☐ Mengidentifikasi antarmuka utama Gazebo (Tombol Play/Pause, Panel Insert, Panel Properties).
- ☐ Menambahkan objek primitif secara manual melalui tab Insert.
- ☐ Mengamati simulasi efek fisika dasar seperti gaya gravitasi dan tumbukan antara objek dan lantai.

Percobaan 2 – Dinamika Fisika pada Shapes World

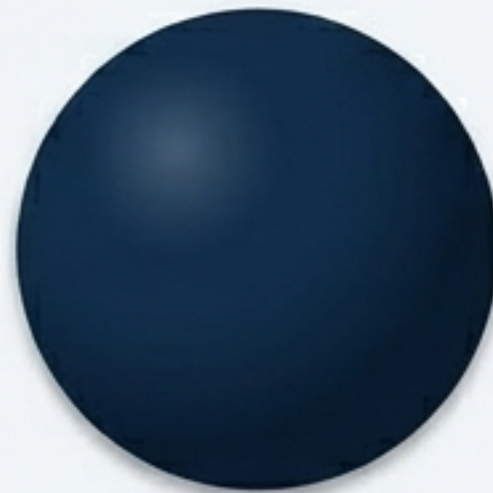


Kebutuhan Sistem:
1 Terminal

```
ros2 launch gazebo_praktikum percobaan2_shapes.launch.py
```



Gravitasi



Penjelasan Eksperimen

- Setiap objek dikonfigurasi dengan properti unik di dalam file SDF: mass, inertia, friction, dan damping.
- **Tugas Praktikan:** Mengamati perbedaan perilaku saat jatuh bebas dan tumbukan.

```
ros2 topic list
```

Gunakan perintah ini untuk melihat daftar topik yang aktif di balik layar Gazebo.

Percobaan 3 – Validasi Model dengan URDF dan RViz2

```
$ ros2 launch gazebo_praktikum percobaan3_urdf_rviz.launch.py
```

Arsitektur Visualisasi (3-Node)

robot_state_publisher

Memproses URDF XML dan me-publish topik /robot_description beserta data /tf.

joint_state_publisher_gui

Menampilkan slider GUI interaktif untuk memanipulasi sendi (joint).

rviz2

Merender model 3D berdasarkan perhitungan frame koordinat aktual.



Tugas Praktikan: Menggerakkan slider di GUI dan memantau bagaimana model robot dan hierarki TF tree berubah secara real-time.

Percobaan 4 – Spawn Robot ke dalam Simulasi Gazebo

 **Kebutuhan Sistem:** 1 Terminal

```
$ ros2 launch gazebo_praktikum percobaan4_spawn_robot.launch.py
```

Mekanisme Delay (TimerAction)

Script `spawn_entity.py` diatur dengan delay selama 2 detik. Hal ini vital agar core engine 2 detik. Hal ini vital agar core engine Gazebo memiliki waktu booting sebelum objek dimasukkan, mencegah crash.

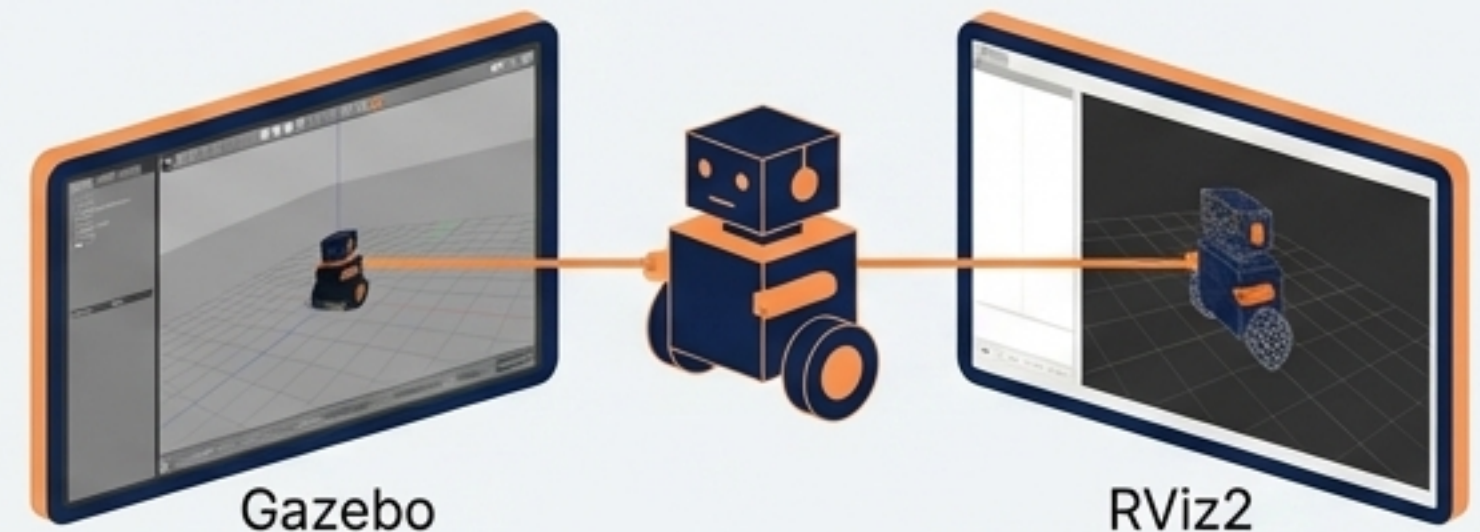
Argumen Parameter Node:

- entity robot_sederhana
- topic /robot_description

Koordinat Posisi Awal Z-axis dinaikkan: (0, 0, 0.05).

Hasil Sinkronisasi

Sinkronisasi visualisasi—robot merender bentuknya secara bersamaan di dunia fisika (Gazebo) dan panel inspeksi (RViz2).



Kebutuhan Sistem:
2 Terminal (Simulasi
& Kontrol)

Percobaan 5 – Uji Coba Sensor Kamera dan LIDAR

```
$ ros2 launch gazebo_praktikum  
percobaan5_sensor.launch.py
```

```
$ ros2 run teleop_twist_keyboard  
teleop_twist_keyboard
```

Checklist Monitoring Topik RViz2

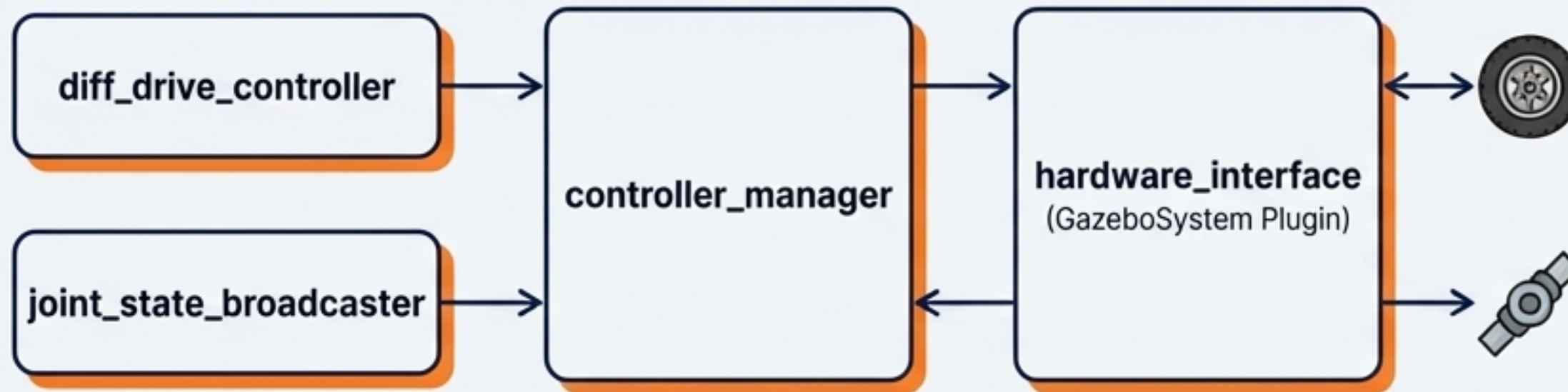
- ☐ /scan (Tampilan pola titik LIDAR)
- ☐ /robot/camera/image_raw & /robot/camera/camera_info (Panel gambar visual)
- ☐ /odom (Posisi pergerakan roda)

Debugging Command

```
$ ros2 topic hz /scan
```

Gunakan untuk memeriksa kestabilan frekuensi hertz pengiriman data sensor.

Arsitektur Standar Kontrol Robot (ros2_control)



Komponen 1: controller_manager
(Satu node sentral yang memuat dan manajemen semua kontroler internal).

Komponen 2: Controller Aktif
(diff_drive_controller untuk roda & joint_state_broadcaster untuk publish status sendi).

Komponen 3: hardware_interface
(Abstraksi yang menjembatani kontroler dengan fisik atau simulasi melalui plugin GazeboSystem).

🔧 Deskripsi Tambahan: Tersedia kontroler lain seperti joint_trajectory_controller untuk pengaturan pergerakan lengan mekanis (manipulator).

Konfigurasi YAML dan Spawner Node (ros2_control)

ros2_control_diff_drive.yaml

```
update_rate: 100 Hz

joint_state_broadcaster
  (Type: JointStateBroadcaster)

diff_drive_controller
  (Type: DiffDriveController)
  (Parameter Kunci: wheel_separation: 0.2,
  wheel_radius: 0.05, cmd_vel_timeout: 0.5)
```

Spawner Command

```
$ ros2 run controller_manager
spawner diff_drive_controller
```



Spawning node ini tidak dilakukan secara bersamaan. Di dalam launch file, semua spawner harus menggunakan metode penundaan bertingkat (staggered TimerAction delay) guna mengamankan transisi status sebelum controller berikutnya dimuat.